FIFO PROJECT

By:
Menna Allah Mohamed

DESIGN BEFORE EDITS

```
////////////////////////////////////////////////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
//
// Description: FIFO Design
//
////////////////////////////////////////////////////////////////////////////////
module FIFO(data_in, wr_en, rd_en, clk, rst_n, full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
input [FIFO_WIDTH-1:0] data_in;
input clk, rst_n, wr_en, rd_en;
output reg [FIFO_WIDTH-1:0] data_out;
output reg wr_ack, overflow;
output full, empty, almostfull, almostempty, underflow;

localparam max_fifo_addr = $clog2(FIFO_DEPTH);

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_ptr <= 0;
    end
    else if (wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= data_in;
        wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        wr_ack <= 0;
        if (full & wr_en)
            overflow <= 1;
        else
            overflow <= 0;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_ptr <= 0;
    end
    else if (rd_en && count != 0) begin
        data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({wr_en, rd_en} == 2'b10) && !full)
            count <= count + 1;
        else if ( ({wr_en, rd_en} == 2'b01) && !empty)
            count <= count - 1;
    end
end

assign full = (count == FIFO_DEPTH)? 1 : 0;
assign empty = (count == 0)? 1 : 0;
assign underflow = (empty && rd_en)? 1 : 0;
assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
assign almostempty = (count == 1)? 1 : 0;

endmodule
```

DESIGN AFTER EDITS

```
module FIFO(FIFO_if.DUT f);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
localparam max_fifo_addr = $clog2(FIFO_DEPTH);

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge f.clk or negedge f.rst_n) begin
    if (!f.rst_n) begin
        wr_ptr <= 0;
        f.overflow <= 0; //reseting overflow
        f.underflow <= 0; //reseting underflow
        f.wr_ack <= 0; //reseting wr_ack
    end
    else if (f.wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= f.data_in;
        f.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        f.wr_ack <= 0;
        if (f.full & f.wr_en)
            f.overflow <= 1;
        else
            f.overflow <= 0;
    end
end

always @(posedge f.clk or negedge f.rst_n) begin
    if (!f.rst_n) begin
        rd_ptr <= 0;
    end
    else if (f.rd_en && count != 0) begin
        f.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
    else begin
        if (f.empty & f.rd_en)
            f.underflow <= 1;
        else
            f.underflow <= 0;
    end
end

always @(posedge f.clk or negedge f.rst_n) begin
    if (!f.rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({f.wr_en, f.rd_en} == 2'b10) && !f.full)
            count <= count + 1;
        else if ( ({f.wr_en, f.rd_en} == 2'b01) && !f.empty)
            count <= count - 1;
        else if ( ({f.wr_en, f.rd_en} == 2'b11) && f.empty) //handling not reading when empty and writing only
            count <= count + 1;
        else if ( ({f.wr_en, f.rd_en} == 2'b11) && f.full) //hndling not writinf when full and reading only
        count <= count - 1;
    end
end

assign f.full = (count == FIFO_DEPTH)? 1 : 0;
assign f.empty = (count == 0)? 1 : 0;
assign f.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;  //FIFO_DEPTH-1 is 7 so almost full updates
assign f.almostempty = (count == 1)? 1 : 0;
```

SVA

```
// Assertions
`ifdef SIM
    property p_reset;
        @(posedge f.clk) (!f.rst_n) |-> (wr_ptr == 0 && rd_ptr == 0 && count == 0);
    endproperty
    a_reset : assert property(p_reset)
    else $error("Reset behavior fail");
    c_reset : cover property(p_reset);

    property p_wr_ack;
        @(posedge f.clk) disable iff(!f.rst_n) (f.wr_en && !f.full) |=> (f.wr_ack == 1);
    endproperty
    a_wr_ack : assert property(p_wr_ack)
    else $error("wr_ack behavior fail");
    c_wr_ack : cover property(p_wr_ack);

    property p_overflow;
        @(posedge f.clk) disable iff(!f.rst_n) (f.wr_en && f.full) |=> (f.overflow == 1);
    endproperty
    a_overflow : assert property(p_overflow)
    else $error("overflow behavior fail");
    c_overflow : cover property(p_overflow);

    property p_underflow;
        @(posedge f.clk) disable iff(!f.rst_n) (f.rd_en && f.empty) |=> (f.underflow == 1);
    endproperty
    a_underflow : assert property(p_underflow)
    else $error("underflow behavior fail");
    c_underflow : cover property(p_underflow);

    property p_empty;
        @(posedge f.clk) (count == 0) |-> (f.empty == 1);
    endproperty
    a_empty : assert property(p_empty)
    else $error("empty behavior fail");
    c_empty : cover property(p_empty);

    property p_full;
        @(posedge f.clk) (count == FIFO_DEPTH) |-> (f.full == 1);
    endproperty
    a_full : assert property(p_full)
    else $error("full behavior fail");
    c_full : cover property(p_full);

    property p_almostempty;
        @(posedge f.clk) (count == 1) |-> (f.almostempty == 1);
    endproperty
    a_almostempty : assert property(p_almostempty)
    else $error("almostempty behavior fail");
    c_almostempty : cover property(p_almostempty);

    property p_almostfull;
        @(posedge f.clk) (count == FIFO_DEPTH-1) |-> (f.almostfull == 1);
    endproperty
    a_almostfull : assert property(p_almostfull)
    else $error("almostfull behavior fail");
    c_almostfull : cover property(p_almostfull);

    property p_wr_pointer_wraparound;
        @(posedge f.clk) (wr_ptr == FIFO_DEPTH - 1 && f.wr_en && !f.full) |-> ##1 (wr_ptr == 0);
    endproperty
    a_wr_pointer_wraparound : assert property(p_wr_pointer_wraparound)
    else $error("pointer_wr_wraparound behavior fail");
    c_wr_pointer_wraparound : cover property(p_wr_pointer_wraparound);

    property p_rd_pointer_wraparound;
        @(posedge f.clk) (rd_ptr == FIFO_DEPTH - 1 && f.rd_en && !f.empty) |-> ##1 (rd_ptr == 0);
    endproperty
    a_rd_pointer_wraparound : assert property(p_rd_pointer_wraparound)
    else $error("pointer_rd_wraparound behavior fail");
    c_rd_pointer_wraparound : cover property(p_rd_pointer_wraparound);

    property p_pointer_threshold;
        @(posedge f.clk) (wr_ptr < FIFO_DEPTH && rd_ptr < FIFO_DEPTH && count <= FIFO_DEPTH);
    endproperty
    a_pointer_threshold : assert property(p_pointer_threshold)
    else $error("almostfull behavior fail");
    c_pointer_threshold : cover property(p_pointer_threshold);

`endif

endmodule
```

INTERFACE

```
interface FIFO_if(input bit clk);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
logic [FIFO_WIDTH-1:0] data_in;
logic rst_n, wr_en, rd_en;
logic [FIFO_WIDTH-1:0] data_out;
logic wr_ack, overflow;
logic full, empty, almostfull, almostempty, underflow;

modport DUT(
input data_in,rst_n,wr_en,rd_en,clk,
output data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow
);

modport tb(
output data_in,rst_n,wr_en,rd_en,clk,
input data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow
);

modport mon(
input data_in,rst_n,wr_en,rd_en,clk,
data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow
);
endinterface
```

TOP MODULE

```
module top();
bit clk;
initial begin
  forever begin
    #1 clk = ~clk;
  end
end

FIFO_if f(clk);
FIFO DUT(f.DUT);
FIFO_tb tb(f.tb);
FIFO_monitor mon(f.mon);

property async_reset;
  @(negedge f.rst_n) (!f.rst_n) |-> (!f.overflow && !f.underflow && !f.wr_ack);
endproperty
a_async_reset : assert property(async_reset)
else $error("Async reset assertion failed");

endmodule
```

SHARED PACKAGE

```
package shared_pkg;
bit test_finished;
int error_count =0;
int correct_count =0;
event sample_event;

```

TRANSACTION

```
package FIFO_transaction_pkg;
class FIFO_transaction;
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
rand logic [FIFO_WIDTH-1:0] data_in;
rand logic rst_n, wr_en, rd_en;
logic [FIFO_WIDTH-1:0] data_out;
logic wr_ack, overflow;
logic full, empty, almostfull, almostempty, underflow;
int RD_EN_ON_DIST, WR_EN_ON_DIST;

function new(int RD_EN_ON_DIST =30, int WR_EN_ON_DIST=70);
    this.RD_EN_ON_DIST = RD_EN_ON_DIST;
    this.WR_EN_ON_DIST = WR_EN_ON_DIST;
endfunction

constraint const_rst{
rst_n dist {0:/5 , 1:/95};
};
constraint const_wr_en{
wr_en dist {1:/WR_EN_ON_DIST , 0:/(100-WR_EN_ON_DIST)};
};
constraint const_rd_en{
rd_en dist {1:/RD_EN_ON_DIST , 0:/(100-RD_EN_ON_DIST)};
};

endclass
endpackage
```

COVERAGE

```
package FIFO_coverage_pkg;
import FIFO_transaction_pkg::*;
class FIFO_coverage;
    FIFO_transaction F_cvg_txn;
    covergroup CovFIFO;
    c_full : cross F_cvg_txn.wr_en, F_cvg_txn.rd_en, F_cvg_txn.full;
    c_empty : cross F_cvg_txn.wr_en, F_cvg_txn.rd_en, F_cvg_txn.empty;
    c_almost_full : cross F_cvg_txn.wr_en, F_cvg_txn.rd_en, F_cvg_txn.almostfull;
    c_almost_empty : cross F_cvg_txn.wr_en, F_cvg_txn.rd_en, F_cvg_txn.almostempty;
    c_overflow : cross F_cvg_txn.wr_en, F_cvg_txn.rd_en, F_cvg_txn.overflow;
    c_underflow : cross F_cvg_txn.wr_en, F_cvg_txn.rd_en, F_cvg_txn.underflow;
    c_wr_ack : cross F_cvg_txn.wr_en, F_cvg_txn.rd_en, F_cvg_txn.wr_ack;
  endgroup

  function new();
    CovFIFO = new();
  endfunction

  function void sample_data(input FIFO_transaction F_txn);
    F_cvg_txn = F_txn;
    CovFIFO.sample();
  endfunction

endclass

endpackage
```

TEST BENCH

```
import shared_pkg::*;
import FIFO_transaction_pkg::*;
module FIFO_tb(FIFO_if.tb f);
FIFO_transaction txn;
initial begin
    txn = new();
    //reset
    f.rst_n = 0;
    f.wr_en = 0;
    f.rd_en = 0;
    f.data_in = 0;
    @(negedge f.clk);
    -> sample_event;
    f.rst_n = 1;

    repeat(9000) begin
        assert(txn.randomize());
        f.wr_en = txn.wr_en;
        f.rd_en = txn.rd_en;
        f.data_in = txn.data_in;
        f.rst_n = txn.rst_n;
        @(negedge f.clk);
        -> sample_event;
    end

    test_finished =1;

end
endmodule
```

MONITOR

```
import FIFO_transaction_pkg::*;
import FIFO_scoreboard_pkg::*;
import FIFO_coverage_pkg::*;
import shared_pkg::*;

module FIFO_monitor(FIFO_if.mon f);
FIFO_coverage cvr;
FIFO_transaction txn;
FIFO_scoreboard sb;
initial begin
  txn = new();
  sb  = new();
  cvr = new();
  forever begin
    @(negedge f.clk);
    wait(sample_event.triggered);
    txn.rst_n         = f.rst_n;
    txn.wr_en         = f.wr_en;
    txn.rd_en         = f.rd_en;
    txn.data_in       = f.data_in;
    txn.data_out      = f.data_out;
    txn.wr_ack        = f.wr_ack;
    txn.overflow      = f.overflow;
    txn.underflow     = f.underflow;
    txn.full          = f.full;
    txn.empty         = f.empty;
    txn.almostfull    = f.almostfull;
    txn.almostempty   = f.almostempty;

    fork
      begin
        cvr.sample_data(txn);
      end
      begin
        #2;
        sb.check_data(txn);
      end
    join

    if (test_finished) begin
      $display("TEST FINISHED");
      $display("Correct count =
", correct_count);
      $display("Error count   =
", error_count);
    end
  end

end
endmodule
```

SCOREBOARD

```
package FIFO_scoreboard_pkg;
import FIFO_transaction_pkg::*;
import shared_pkg::*;
class FIFO_scoreboard;
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    logic [FIFO_WIDTH-1:0] data_out_ref;
    logic wr_ack_ref, overflow_ref;
    logic full_ref, empty_ref, almostfull_ref, almostempty_ref, underflow_ref;
    bit [FIFO_WIDTH-1:0] mem[$];

    function void check_data(input FIFO_transaction txn);
        if ((data_out_ref != txn.data_out)) begin
            error_count++;
            $display("ERROR @ %0t, data_out_Ref = %h, data_out = %h", $time, data_out_ref, txn.data_out);
            end
        else begin
            correct_count++;
        end
        reference_model(txn);

    endfunction

    function void reference_model(input FIFO_transaction txn);
        wr_ack_ref =0;
        overflow_ref =0;
        underflow_ref =0;
        if(!txn.rst_n) begin
            mem.delete();
        end
        else if(({txn.wr_en, txn.rd_en} == 2'b11) && empty_ref)begin
            mem.push_back(txn.data_in);
            empty_ref = 0;
        end
        else begin
            //write
            if(txn.wr_en) begin
                if(mem.size() < FIFO_DEPTH) begin
                    mem.push_back(txn.data_in);
                    wr_ack_ref =1;
                end
                else begin
                    overflow_ref =1;
                end
            end

            //read
            if(txn.rd_en) begin
                if(mem.size() > 0) begin
                    data_out_ref = mem.pop_front();
                    wr_ack_ref =1;
                end
                else begin
                    underflow_ref =1;
                end
            end
        end

        full_ref = (mem.size() == FIFO_DEPTH)? 1:0;
        empty_ref = (mem.size() == 0)? 1:0;
        almostfull_ref = (mem.size() == FIFO_DEPTH -1)? 1:0;
        almostempty_ref = (mem.size() == 1)? 1:0;
    endfunction
endclass

endpackage
```

DO FILE

```
vlib work
vlog -f src_files.list +cover +define+SIM -covercells
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
coverage save fifo.ucdb -onexit
add wave sim:/top/f/*
add wave -position insertpoint  \
sim:/top/mon/sb.data_out_ref
run -all
```

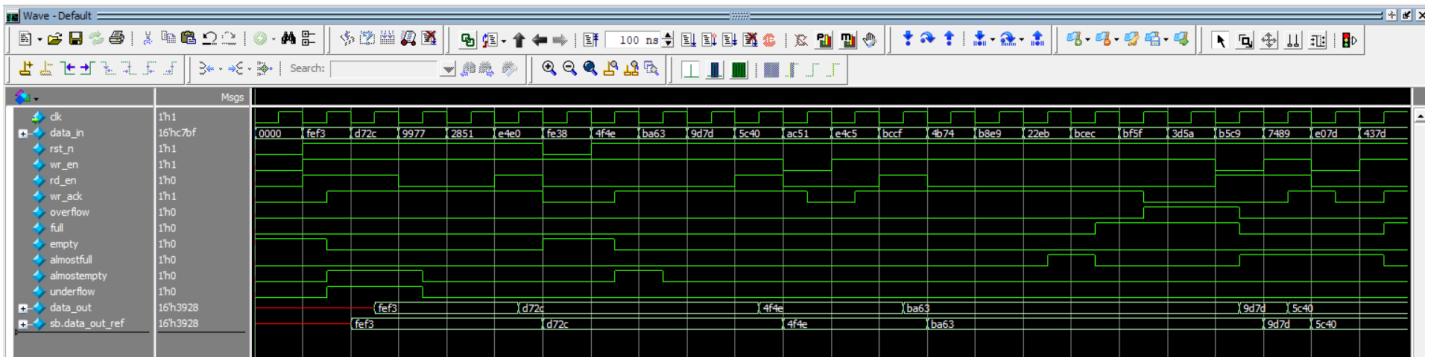
SRC FILES

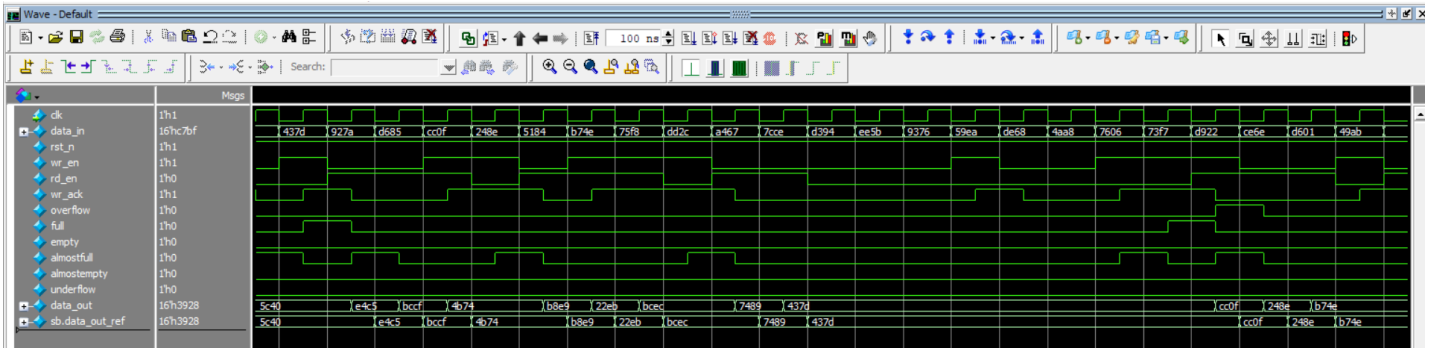
```
shared_pkg.sv
FIFO_transaction.sv
FIFO_coverage.sv
FIFO_scoreboard.sv
FIFO_tb.sv
FIFO_monitor.sv
FIFO_if.sv
FIFO.sv
top_module.sv
```

VERIFICATION PLAN

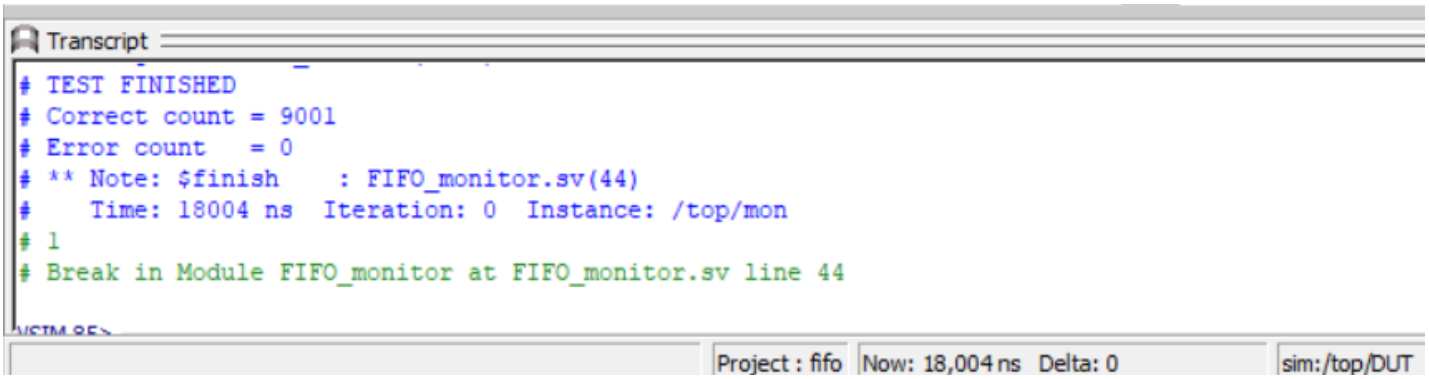
Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
RST_CHECK	Reset behavior for internal signals and some outputs	Assert rst_n low (active low asynchronous reset); verify all internal counters and pointers reset to 0, and empty is asserted		Assertion
Write Operation	Write operation in memory	Assert wr_en when FIFO is not full , verify data is written and internal count increments	cross between wr_en, rd_en, full	Reference Model, Assertion
Read Operation	Make sure that data is retrieved and internal count decrements	Assert rd_en when FIFO is not empty; verify data is retrieved and internal count decrements	cross between wr_en, rd_en, empty	Reference Model, Assertion
R/W Empty	Simultaneous R/W when empty	Assert wr_en=1, rd_en=1 when empty=1; verify only the write succeeds (count increments by 1)	cross between wr_en, rd_en, overflow	Reference Model, Assertion
R/W Full	Simultaneous R/W when full	Assert wr_en=1, rd_en=1 when full=1; verify only the read succeeds (count decrements by 1)	cross between wr_en, rd_en, full, underflow	Reference Model, Assertion
Data_Out Check	Is data_out beging written correctly	Write a sequence of data to the FIFO, followed by a full read sequence, verify data_out matches input sequence in FIFO order		Reference Model
Full Flag	Full flag assertion	Write exactly FIFO_DEPTH entries; verify full asserts when count = FIFO_DEPTH	cross between wr_en, rd_en, full	Assertion
Almost Full	Almost full condition	Write exactly FIFO_DEPTH-1 entries; verify almostfull asserts	cross between wr_en, rd_en, almostfull	Assertion
Empty Flag	Empty flag assertion	Read until count = 0; verify empty asserts	cross between wr_en, rd_en, empty	Assertion
Almost Empty	Almost empty condition	Read until count = 1; verify almostempty asserts	cross between wr_en, rd_en, almostempty	Assertion
Overflow Condition	Overflow detection	Assert wr_en when full is already asserted; verify overflow asserts and write is non-destructive	cross between wr_en, rd_en, overflow	Assertion
Underflow Condition	Underflow detection	Assert rd_en when empty is already asserted; verify underflow asserts and read is non-destructive	cross between wr_en, rd_en, underflow	Assertion
Write Acknowledge	Write acknowledge	Assert wr_en when not full; verify wr_ack asserts for one cycle confirming success	cross between wr_en, rd_en, wr_ack	Assertion
Pointer Wraparound	wr_ptr,rd_ptr,count should wrap around when they reach last index	Drive continuous writes/reads for more than FIFO_DEPTH cycles; check that pointers and counters correctly wrap around from depth to 0 and never exceed FIFO_DEPTH		Assertion
Pointer Threshold	wr_ptr,rd_ptr pointers cannot exceed the FIFO_DEPTH entries as well as the counter	Drive continuous writes/reads for more than FIFO_DEPTH cycles; check that pointers and counters never exceed FIFO_DEPTH		Assertion

WAVEFORM SNIPPETS





TRANSCRIPT



ASSERTIONS

Assertions

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak M
/top/a_async_reset	Concurrent	SVA	on	0	0	-	0B	
/top/DUT/a_reset	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_wr_ack	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_overflow	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_underflow	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_empty	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_full	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_almostempty	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_almostfull	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_wr_pointer_wra...	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_rd_pointer_wra...	Concurrent	SVA	on	0	1	-	0B	
/top/DUT/a_pointer_thresho...	Concurrent	SVA	on	0	1	-	0B	
/top/tb/#ublk#182146786#...	Immediate	SVA	on	0	1	-	-	

COVER DIRECTIVES

Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memo
/top/DUT/c_reset	SVA	✓	Off	421	1	Unli...	1	100%		✓	
/top/DUT/c_wr_ack...	SVA	✓	Off	4356	1	Unli...	1	100%		✓	
/top/DUT/c_overflo...	SVA	✓	Off	1465	1	Unli...	1	100%		✓	
/top/DUT/c_underfl...	SVA	✓	Off	171	1	Unli...	1	100%		✓	
/top/DUT/c_empty	SVA	✓	Off	1054	1	Unli...	1	100%		✓	
/top/DUT/c_full	SVA	✓	Off	2183	1	Unli...	1	100%		✓	
/top/DUT/c_almost...	SVA	✓	Off	796	1	Unli...	1	100%		✓	
/top/DUT/c_almost...	SVA	✓	Off	1691	1	Unli...	1	100%		✓	
/top/DUT/c_wr_poi...	SVA	✓	Off	402	1	Unli...	1	100%		✓	
/top/DUT/c_rd_poi...	SVA	✓	Off	169	1	Unli...	1	100%		✓	
/top/DUT/c_pointer...	SVA	✓	Off	9002	1	Unli...	1	100%		✓	

COVERGROUPS

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_i
/FIFO_coverage_pkg/FIFO_coverage		100.00%					
TYPE CovFIFO		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_e...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.rd_en...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_ac...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_e...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.rd_en...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.under...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_e...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.rd_en...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.overfl...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_e...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.rd_en...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.almos...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_e...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.rd_en...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.almos...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_e...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.rd_en...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.empt...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.wr_e...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.rd_en...		100.00%	100	100.00...			
CVP CovFIFO::{#F_cvg_txn.full__...		100.00%	100	100.00...			
CROSS CovFIFO::c_full		100.00%	100	100.00...			
CROSS CovFIFO::c_empty		100.00%	100	100.00...			
CROSS CovFIFO::c_almost_full		100.00%	100	100.00...			
CROSS CovFIFO::c_almost_empty		100.00%	100	100.00...			
CROSS CovFIFO::c_overflow		100.00%	100	100.00...			
CROSS CovFIFO::c_underflow		100.00%	100	100.00...			
CROSS CovFIFO::c_wr_ack		100.00%	100	100.00...			

CODE COVERAGE ANALYSIS

Toggle :

Code Coverage Analysis

Toggles - by instance (/top/DUT) Toggle

```
sim:/top/DUT
  count
  rd_ptr
  wr_ptr
```

Statement :

Code Coverage Analysis

Statements - by instance (/top/DUT) Statement

```
FIFO.sv
18 always @(posedge f.clk or negedge f.rst_n) begin
20 wr_ptr <= 0;
21 f.overflow <= 0; //reseting overflow
22 f.underflow <= 0; //reseting underflow
23 f.wr_ack <= 0; //reseting wr_ack
26 mem[wr_ptr] <= f.data_in;
27 f.wr_ack <= 1;
28 wr_ptr <= wr_ptr + 1;
31 f.wr_ack <= 0;
33 f.overflow <= 1;
35 f.overflow <= 0;
39 always @(posedge f.clk or negedge f.rst_n) begin
41 rd_ptr <= 0;
44 f.data_out <= mem[rd_ptr];
45 rd_ptr <= rd_ptr + 1;
49 f.underflow <= 1;
51 f.underflow <= 0;
55 always @(posedge f.clk or negedge f.rst_n) begin
57 count <= 0;
61 count <= count + 1;
63 count <= count - 1;
65 count <= count + 1;
67 count <= count - 1;
71 assign f.full = (count == FIFO_DEPTH)? 1 : 0;
72 assign f.empty = (count == 0)? 1 : 0;
73 assign f.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;  //FIFO_DEPTH-1 is 7 so al...
74 assign f.almostempty = (count == 1)? 1 : 0;
```

Branch :

Code Coverage Analysis

Branches - by instance (/top/DUT) Branch

FIFO.sv

```
19 if (!f.rst_n) begin
25 else if (f.wr_en && count < FIFO_DEPTH) begin
30 else begin
32 if (f.full & f.wr_en)
34 else
40 if (!f.rst_n) begin
43 else if (f.rd_en && count != 0) begin
47 else begin
48 if (f.empty & f.rd_en)
50 else
56 if (!f.rst_n) begin
59 else begin
60 if       ( ({f.wr_en, f.rd_en} == 2'b10) && !f.full)
62 else if ( ({f.wr_en, f.rd_en} == 2'b01) && !f.empty)
64 else if ( ({f.wr_en, f.rd_en} == 2'b11) && f.empty) //handling not reading when ...
66 else if ( ({f.wr_en, f.rd_en} == 2'b11) && f.full) //hndling not writinf when fu...
71 assign f.full = (count == FIFO_DEPTH)? 1 : 0;
72 assign f.empty = (count == 0)? 1 : 0;
73 assign f.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;  //FIFO_DEPTH-1 is 7 so al...
74 assign f.almostempty = (count == 1)? 1 : 0;
```

COVERAGE REPORT

Statement coverage :

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      27        27         0   100.00%

================================Statement Details================================

Statement Coverage for instance /top/DUT --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO.sv
    8                                                module FIFO(FIFO_if.DUT f);

    9                                                parameter FIFO_WIDTH = 16;

    10                                               parameter FIFO_DEPTH = 8;

    11                                               localparam max_fifo_addr = $clog2(FIFO_DEPTH);

    12

    13                                               reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];

    14

    15                                               reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;

    16                                               reg [max_fifo_addr:0] count;

    17

    18             1                          9408   always @(posedge f.clk or negedge f.rst_n) begin

    19                                                  if (!f.rst_n) begin

    20             1                           827           wr_ptr <= 0;

    21             1                           827           f.overflow <= 0; //reseting overflow

    22             1                           827           f.underflow <= 0; //reseting underflow

    23             1                           827           f.wr_ack <= 0; //reseting wr_ack
```

Toggle coverage :

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         20        20         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /top/DUT --

                                            Node      1H->0L      0L->1H  "Coverage"
                                            --------------------------------------
                                       count[3-0]           1           1      100.00
                                      rd_ptr[2-0]           1           1      100.00
                                      wr_ptr[2-0]           1           1      100.00

Total Node Count     =         10
Toggled Node Count   =         10
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (20 of 20 bins)
```

Branch Coverage:

```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        25        25         0   100.00%

================================Branch Details================================

Branch Coverage for instance /top/DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO.sv
------------------------------------IF Branch------------------------------------
    19                                      9408     Count coming in to IF
    19              1                        827        if (!f.rst_n) begin

    25              1                       4561        else if (f.wr_en && count < FIFO_DEPTH) begin

    30              1                       4020        else begin

Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    32                                      4020     Count coming in to IF
    32              1                       1545            if (f.full & f.wr_en)

    34              1                       2475            else

Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    40                                      9408     Count coming in to IF
    40              1                        827        if (!f.rst_n) begin

    43              1                       2403        else if (f.rd_en && count != 0) begin

    47              1                       6178        else begin

Branch totals: 3 hits of 3 branches = 100.00%

```

Assertion coverage :

```
================================================================================
=== Instance: /top/DUT
=== Design Unit: work.FIFO
================================================================================

Assertion Coverage:
    Assertions                      11        11         0   100.00%
----------------------------------------------------------------------
Name                 File(Line)                   Failure      Pass
                                                  Count        Count
----------------------------------------------------------------------
/top/DUT/a_reset     FIFO.sv(81)                       0          1
/top/DUT/a_wr_ack    FIFO.sv(88)                       0          1
/top/DUT/a_overflow  FIFO.sv(95)                       0          1
/top/DUT/a_underflow FIFO.sv(102)                      0          1
/top/DUT/a_empty     FIFO.sv(109)                      0          1
/top/DUT/a_full      FIFO.sv(116)                      0          1
/top/DUT/a_almostempty
                     FIFO.sv(123)                      0          1
/top/DUT/a_almostfull
                     FIFO.sv(130)                      0          1
/top/DUT/a_wr_pointer_wraparound
                     FIFO.sv(137)                      0          1
/top/DUT/a_rd_pointer_wraparound
                     FIFO.sv(144)                      0          1
/top/DUT/a_pointer_threshold
                     FIFO.sv(151)                      0          1
```

Directive coverage :

```
Directive Coverage:
    Directives                    11        11         0   100.00%

DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                    Design Design   Lang File(Line)      Hits Status
                                        Unit   UnitType
--------------------------------------------------------------------------------------------
/top/DUT/c_reset                        FIFO   Verilog  SVA  FIFO.sv(83)      421 Covered
/top/DUT/c_wr_ack                       FIFO   Verilog  SVA  FIFO.sv(90)     4356 Covered
/top/DUT/c_overflow                     FIFO   Verilog  SVA  FIFO.sv(97)     1465 Covered
/top/DUT/c_underflow                    FIFO   Verilog  SVA  FIFO.sv(104)     171 Covered
/top/DUT/c_empty                        FIFO   Verilog  SVA  FIFO.sv(111)    1054 Covered
/top/DUT/c_full                         FIFO   Verilog  SVA  FIFO.sv(118)    2183 Covered
/top/DUT/c_almostempty                  FIFO   Verilog  SVA  FIFO.sv(125)     796 Covered
/top/DUT/c_almostfull                   FIFO   Verilog  SVA  FIFO.sv(132)    1691 Covered
/top/DUT/c_wr_pointer_wraparound        FIFO   Verilog  SVA  FIFO.sv(139)     402 Covered
/top/DUT/c_rd_pointer_wraparound        FIFO   Verilog  SVA  FIFO.sv(146)     169 Covered
/top/DUT/c_pointer_threshold            FIFO   Verilog  SVA  FIFO.sv(153)    9002 Covered
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      27        27         0   100.00%
```